Magnus Armsen (A Robotic Chess Playing Arm)

Students Names

Hazem Mohamed - 211001615
Mohamed Youssef - 211001821
Belal Sameh - 211000411
Mohamed Abdelraheem - 211002827

Course

GN312 - Embedded Sytems & IoT

Dr. Omar Shalash
Eng. Hossam Eldeen

Contents

Introduction

Summary

- The main purpose of the project is to build a fully automated robotic arm that can play chess against a human player using the stockfish chess engine.
- The expected result of the project is to have a robotic arm with 6 degrees of freedom that can detect the user's movement on the chess board using various image processing technologies and perform the best move for the current chess position.
- The total anticipated budget is 9000 EGP.

Motives

- After careful consideration, we realized that developing a robotic arm capable of playing chess was not a common undertaking. However, we were enticed by the prospect of a project that would simultaneously test our coding skills and hardware proficiency. It became evident that this project was the ideal fit for our team.
- Organizing a chess tournament on our campus, offering an opportunity for the winner to compete against our robotic arm. This tournament aims to showcase the capabilities of our advanced robotic arm, designed specifically for playing chess. By organizing this event, we seek to engage chess enthusiasts and provide them with a platform to test their skills against our technology. The tournament promises to be an exciting and competitive occasion, serving as a testament to the progress achieved in the field of robotics.

Features

Interface

- A web application has been developed to provide users with the opportunity to engage with our project in various ways. The app features a short description of the project, as well as links to explore additional information about the team. The main highlight of the app is the play page, where users can challenge and compete against Magnus Armsen, our remarkable robotic arm. This interactive platform allows users to explore a multitude of options, such as taking control of the pieces directly on the page using the integrated chess board instead of physically moving them on the board. Additionally, users can request a draw, resign from the game, or even engage in a chat with Magnus himself.

- Through the development of this web application, we aim to showcase our proficiency in the field of IoT, demonstrating our ability to integrate advanced technologies into an engaging and immersive chess-playing experience.

Camera

- The Raspberry Pi official camera has been employed to detect the user's movements of the chess pieces on the physical board. These movements are detected through the application of various camera vision techniques, including wrapping, pose detection, and homography matrix detection between two images.

- This implementation effectively demonstrates our team's proficiency in image detection and camera vision techniques.

Chess engine

- We utilized the Stockfish chess engine, which assumes the role of Magnus Armsen, to determine the optimal chess move according to the user's selected difficulty level and the present chess position.

○ Robotic Arm

- The team 3D-printed the individual pieces of the arm, assembled them with servo motors, and connected them using the I2C protocol on a Raspberry Pi 4. This robotic arm serves as the central component of our project, enabling it to move chess pieces per the optimal moves determined by the Stockfish engine. It possesses the capability to either move its pieces or capture the opponent's pieces based on the best move.

- By connecting the robotic arm to a Raspberry Pi and facilitating the movement of chess pieces across the physical board, we aim to demonstrate our abilities in both robotics and hardware construction fields.

Hardware Specifications

○ Raspberry Pi 4b 8GB

- The Raspberry Pi is a microcontroller that runs on Linux which allows its supported operating system, Pi OS, to be open source and run a suite of open-source software, it also provides a set of GPIO (general purpose input/output) pins, allowing the user to control electronic components for physical computing and Internet of Things (IoT) usage.

Processor	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8)64-bit SoC @ 1.5GHz
Memory	8GB LPDDR4
Connectivity	2.4 GHz and 5.0 GHz IEEE wireless LAN, Bluetooth 5.0, BLE Gigabit Ethernet
GPIO	Standard 40-pin GPIO header
Input power	5V DC via USB-C connector (minimum 3.1 A) 5V DC via GPIO header (minimum 3.1 A) Power over Ethernet (PoE)

○ Raspberry Pi HQ Camera

- The official Raspberry Pi camera.

Sensor	Sony IMX477R stacked, back-illuminated 7.9 mm sensor diagonal
Resolution	12.3 megapixels
HD Video recording	1080p at 30fps, 720p at 60fps, 960p at 45fps
Interface	CSI (Camera Serial Interface)

○ Mg996r Servo

- High Torque Metal Gear Dual Ball Bearing Servo

Stall torque	4 kgf·cm (4.8 V), 11 kgf·cm (6 V)
Operating speed	0.17 s/60º (4.8 V), 0.14 s/60º (6 V)
Operating voltage	4.8 V - 7.2 V
Running Current	500 mA – 900 mA
Temperature range	0 ºC – 55 ºC
Rotation	120º (60 in each direction)

○ Sg90 Servo

- Tiny and lightweight servo with high output power

Stall torque	1.8 kgf·cm (4.8 V)
Operating speed	0.1s/60º (4.8 V)
Operating voltage	4.8 V

Running Current	100 mA – 500 mA
Temperature range	0 °C – 55 °C
Rotation	180° (90 in each direction)

○ CYS-S0200 Servo

- CYS-S0200 20Kg Digital Metal Steering Gear Servo Motor

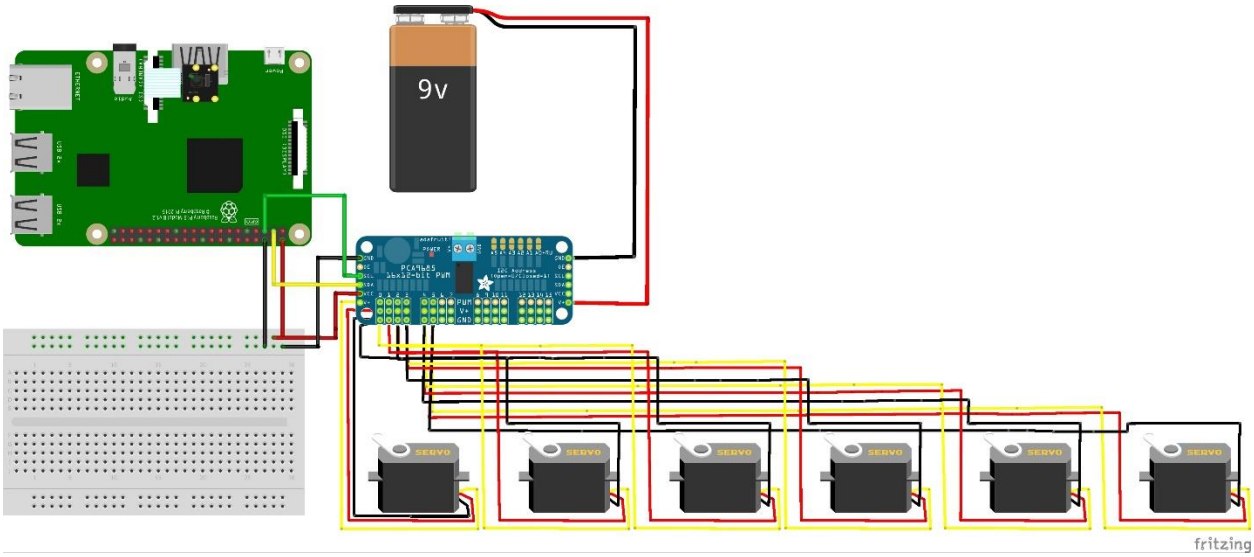
Stall torque	18 kgf·cm (4.8 V), 21.5 kgf·cm (6.8 V)
Operating speed	0.16 s/60°(5V), 0.14 s/60°(6.8V)
Operating voltage	4.8 V – 6.8 V
Running Current	2100 mA – 2700 mA (20 kg)
Temperature range	-15 °C – 70 °C
Rotation	270° (135 in each direction)

○ 16 Channel 12-bit PWM Servo Driver – PCA9685

- Expandable five-channel bidirectional buffer for I2C and SMBus applications

Supply voltage range	-0.5 V – 0.7 V
I2C bus voltage range	-0.5 V – 0.7 V
Operating voltage	3 V – 5 V
Temperature range	-40 °C – 85 °C

○ Wiring and schematics



Software Implementation in Python

○ Necessary Libraries

- Adafruit: The library that is used to control the servo I2C that takes control of the servo motors in the robotic arm.
- OpenCV: The library is used for the camera and chessboard detection.
- Numpy: The library that is used for image matrix manipulation.
- Flask: The framework used for running the backend server and handling all the web page requests and responses.
- Chess: The library that is used for pushing the chess moves that are done by both the engine and the player.
- Stockfish: The library that is used to control the chess engine and find the best moves.

○ C2M (Chess Board to Moves)

Module Name	Brief	Arguments	Returns
showImg	Show an image on the screen	img (ndarray) : An image windowName (string) : Name of the window	
findHomography	Finds the homography matrix between an image of the current board and an image of the ideal board positions	img (ndarray) : The current image of the empty chess board in BGR color format	homography (ndarray): Homography matrix between frame and cbPattern
applyHomography	Warps the current image of the board	img (ndarray) : The current image of the full chess board in BGR color format H (ndarray) : Homography matrix	imgNew (ndarray): Warped image of the full chessboard
findMoves	Finds the chessboard moves of the most changed 4, 3, or 2 squares	img1 (ndarray) : The Rotated warped image of the full chess board before the moves were played img2 (ndarray) : The Rotated warped image of the full chess board after the moves were played	moves (array) : Chessboard moves of the most squares confidenceRate (array): Confidence rate for each move
takePic	Takes a 640x480 picture in bgr format		frame (ndarray): Picture taken

○ ARM (Controls arm's movements)

Module Name	Brief	Arguments	Returns
servo_class	Initializes the servo motor with a channel and an offset	Channel (int) : The channel for the servo motor on the PCA9685 ZeroOffset (int) : The reference position of the servo motor	
servo_class.Setpos	Sets the servo's angle of rotation	pos (int) : The pulse value, 150~650 pulses correspond to 0°~180° angles	
Goto	Moves the arm to a chessboard location	move (string) : The chessboard square index	
grab	Closes the gripper servo which corresponds to grabbing a chess piece		
drop	Opens the gripper servo which corresponds to dropping a chess piece		
initial	Sets the arm to the initial position from a chessboard location	move (string) : The chessboard square index	
gotobox	Moves the arm to the taken chess pieces box		
gofrombox	Returns the arm from the taken chess pieces box		

smooth	Moves a servo from an initial degree to another degree	servo (servo_class object): The specified servo that will change its degree start (int): The servo's initial rotation angle end (int): The servo's final rotation angle total_delay (int): The total time the servo will take in seconds to move	
moveArm	The main function which moves the arm from a chessboard square to another based on the algebraic chess notation	moves (string): The algebraic chess notation kill (bool): Specifies if a chess piece takes another piece in the current turn	

- app (The main app which runs the back end)

Module Name	Brief	Arguments	Returns
app.route("/") home	Renders the home.html file		
app.route("/links") links	Renders the links.html file		
app.route("/play") play	Renders the play.html file		
app.route("/update") send	Replies to the get requests requested by the front-end		

app.route("/process/<string:data>/<string:name>") process	Processes the post requests sent by the front-end		
calibrate	Calibrates the chessboard's corners		
setDifficulty	Sets the engine difficulty		
processPrevImg	Applies wrapping on the img before the user plays his move		
processCurImg	Applies wrapping on the img after the user plays his move		
ACN	Finds the algebraic chess notation based on a list of guessed moves	moves (string): The algebraic chess notation	
reset	Resets all the variables to their initial states when the play page is rendered		
checkOutcome	Checks if the game has ended		bool (bool): True if the game has ended and false otherwise
playing	Moves the arm based on the best move that is retrieved from the engine while checking the physical board for moves		
played	Moves the arm based on the best move that is retrieved from the engine without checking the physical board for moves		

Budget Sheet

Component	Quantity	Price
Raspberry Pi 4 Model B – 8GB	1	5,600 EGP
Raspberry Pi HQ Camera	1	850 EGP
Mg996r Servo	2	190 EGP
Sg90 Servo	3	80 EGP
16 Channel 12-bit PWM Servo Driver – PCA9685	1	132 EGP
3D Printer Filament PLA+ eSUN 1.75mm Skin	1	675 EGP
CYS-S0200 20Kg Digital Metal Steering Gear Servo Motor	1	550 EGP
Miscellaneous	1	200 EGP
		Total: 8,627 EGP

Project Timeline

Phases	Description of Work	Start and End Dates
Phase One	Brainstorming	11/3/2023 till 20/3/2023
Phase Two	Developing the Code and Setting up the Hardware	21/3/2023 till 1/5/2023
Phase Three	Code debugging and Project Testing	2/5/2023 till 28/5/2023

Future Plans

- Implementing a database that records all games played against a specific user, which can also be retrieved upon user request, which allows the user to review older games.
- Adding a voice control feature, enabling users to verbally command the arm for executing moves. This innovative addition allows the user to dictate their desired moves, while also facilitating the arm to simulate both the user's and the arm’s moves.
- Additionally, incorporating a feature that enables the arm to rectify incorrect moves made by the user, thus significantly reducing the margin for human error.
- Also, we are focused on implementing celebratory and mourning gestures that the robotic arm can perform once the game concludes.
- Allowing other users to spectate if a game has already started and even comment on both the user’s and Magnus’s moves.

References

- Chess & Stockfish in Python: https://www.youtube.com/watch?v=iEaU__JdI7c
- Stockfish: https://stockfishchess.org/
- Chess Interface reference: https://www.chess.com/
- Assembling the Arm: DIY Arduino Robot Arm with Smartphone Control - How To Mechatronics
- Full Arm 3D Model: Robotic Arm 3D Model - 3D model by HowToMechatronics on Thangs
- 3D Printing Mass Calculator: https://alanderossett.com/3d/
- Robotic arm simulator: Robotic Arm Simulator – GeoGebra
- Robotic arm documentation: Robotic_Arm_Tutorial_Word.docx-rev1.pdf
- Chess notation explanation: https://www.chess.com/terms/chess-notation
- FEN explanation: https://www.chess.com/terms/fen-chess
- Pose Estimation (Chessboard corners detection): https://www.fritz.ai/pose-estimation/#part-basics